

Assignment-3

Virtualization and Cloud Computing (CSL7510)

Mohit Meemrauth

B23CS1038

Abstract

This project presents a hybrid auto-scaling system that integrates a local virtual machine with Google Cloud Platform (GCP) to dynamically provision resources based on real-time CPU utilization. The system leverages Prometheus for monitoring, Node Exporter for metric collection, and a Python-based automation layer to implement a feedback-driven control loop.

When the system detects that CPU utilization exceeds a predefined threshold, it automatically triggers the creation of a new virtual machine instance in GCP using the gcloud CLI. This architecture demonstrates key cloud-native principles such as observability, decoupled system design, and automated infrastructure provisioning.

The implementation provides a practical foundation for understanding real-world auto-scaling mechanisms and can be extended with alerting systems, load balancing, and distributed workload management.

Project Demonstration Video:

<https://www.youtube.com/watch?v=eZl81zTktb8>

Source Code Repository:

<https://github.com/Mohi1038/automation-scripts>

Contents

1	Introduction	3
2	System Architecture	3
2.1	Layered Design	3
2.2	High-Level Software Architecture	3

2.3	Low-Level Architecture	3
3	Local VM Setup	5
3.1	Configuration	5
3.2	Environment Setup	5
4	Monitoring using psutil	5
4.1	Implementation	5
4.2	Technical Explanation	5
5	GCP Integration	6
5.1	Authentication	6
5.2	Verification	6
6	Auto-Scaling Mechanism	6
6.1	VM Provisioning Script	6
6.2	Technical Detail	6
7	Prometheus Monitoring	7
7.1	System Role and Architecture	7
7.2	Node Exporter Integration	7
7.3	Prometheus Scrape Configuration	7
7.4	Internal Data Flow	7
8	CPU Utilization Query	8
8.1	PromQL Expression	8
8.2	Metric Semantics	8
8.3	Step-by-Step Breakdown	8
8.4	Mathematical Interpretation	8
8.5	Practical Considerations	8
9	Automation via Prometheus API	9
9.1	Automation Sequence Flow	9
10	Application Deployment	9
11	Results	10
12	Conclusion	10
13	Plagiarism Declaration	10

1 Introduction

This project implements a hybrid auto-scaling architecture where a local virtual machine dynamically provisions cloud resources upon exceeding a predefined utilization threshold. The system integrates observability, decision logic, and infrastructure provisioning into a unified pipeline.

Core Idea: Monitoring → Threshold Detection → Automated Cloud Scaling

2 System Architecture

2.1 Layered Design

Layer	Function
Monitoring Layer	Collects system metrics (CPU, memory)
Decision Layer	Evaluates threshold conditions
Infrastructure Layer	Provisions cloud resources

2.2 High-Level Software Architecture

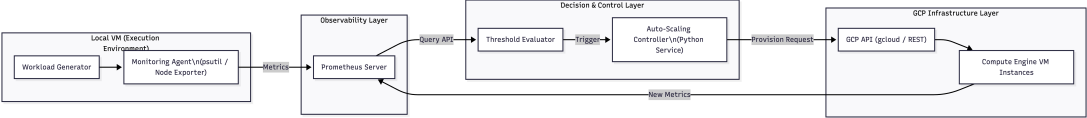


Figure 1: Software Engineering View of Hybrid Auto-Scaling System

Key Insight: The architecture follows a feedback-driven control system where monitoring, decision-making, and infrastructure provisioning are decoupled into independent components communicating via APIs.

2.3 Low-Level Architecture

Technical Insight: This diagram captures the internal working of Prometheus, explicit API interactions, and the execution path from metric generation to automated cloud provisioning.

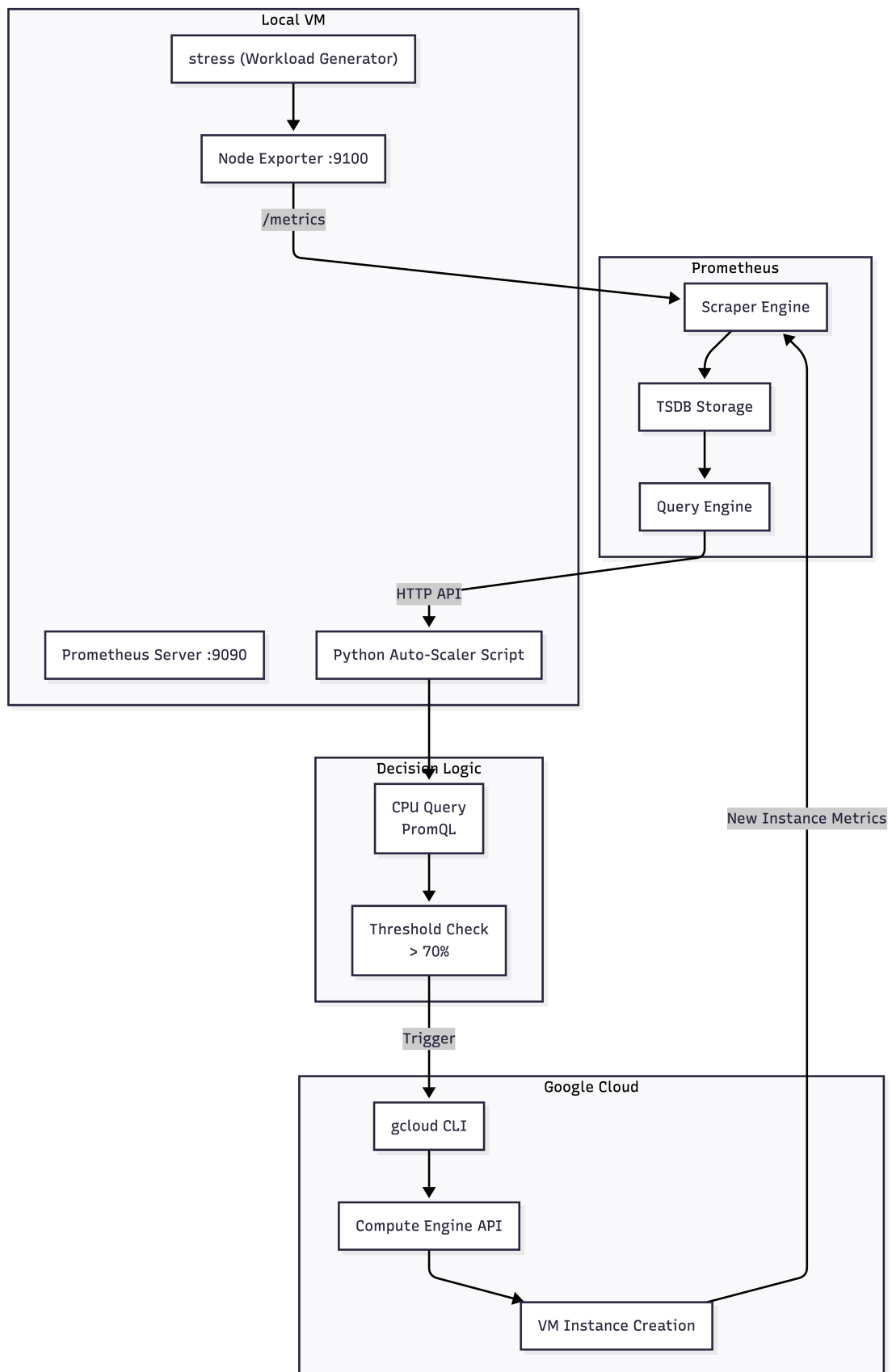


Figure 2: Low-Level Architecture with Internal Components and Execution Flow

3 Local VM Setup

3.1 Configuration

OS: Ubuntu 24.04
RAM: 2GB
CPU: 2 cores
Disk: 20GB

3.2 Environment Setup

```
sudo apt update
sudo apt install python3 python3-pip stress -y
```

Purpose:

- Python: automation layer
- pip: dependency management
- stress: synthetic workload generation

4 Monitoring using psutil

4.1 Implementation

```
import psutil
cpu = psutil.cpu_percent(interval=1)
```

4.2 Technical Explanation

psutil retrieves metrics from kernel interfaces such as:

- /proc/stat → CPU statistics
- /proc/meminfo → memory usage

Observation: Direct system-level monitoring provides low-latency measurement.

5 GCP Integration

5.1 Authentication

```
gcloud init
```

5.2 Verification

```
gcloud config list
```

Authenticated environment ensures secure API interaction.

6 Auto-Scaling Mechanism

6.1 VM Provisioning Script

```
import os, time
vm_name = f"auto-scale-vm-{int(time.time())}"
```

6.2 Technical Detail

The command:

```
gcloud compute instances create
```

invokes Compute Engine APIs to allocate:

- Virtual CPU
- Memory
- Persistent disk
- Network interface

Constraint: Instance names must be globally unique within a zone.

7 Prometheus Monitoring

7.1 System Role and Architecture

Prometheus operates as a pull-based observability system responsible for collecting, storing, and querying time-series metrics generated by the local virtual machine and subsequently provisioned cloud instances.

Design Principle: Decoupled monitoring ensures that metric collection is independent of application logic, improving system reliability and scalability.

7.2 Node Exporter Integration

Node Exporter runs as a lightweight daemon exposing system-level metrics via an HTTP endpoint:

```
http://localhost:9100/metrics
```

Internally, Node Exporter reads from kernel interfaces:

- `/proc/stat` → CPU time slices
- `/proc/meminfo` → memory allocation
- `/proc/loadavg` → system load averages

7.3 Prometheus Scrape Configuration

Prometheus periodically scrapes the Node Exporter endpoint using the following configuration:

```
scrape_configs:  
  - job_name: 'node_exporter'  
    static_configs:  
      - targets: ['localhost:9100']
```

7.4 Internal Data Flow

- **Scraper Engine:** Pulls metrics at fixed intervals
- **TSDB (Time-Series Database):** Stores timestamped metric samples
- **Query Engine:** Executes PromQL queries over stored data

Observation: The pull-based model eliminates dependency on push agents and ensures fault tolerance during transient failures.

8 CPU Utilization Query

8.1 PromQL Expression

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

8.2 Metric Semantics

The metric `node_cpu_seconds_total` represents cumulative CPU time spent in different modes (idle, user, system, etc.). Since it is monotonically increasing, derivative functions are required for real-time analysis.

8.3 Step-by-Step Breakdown

1. **Filtering:** `mode="idle"` isolates idle CPU time.
2. **Rate Calculation:** `rate(... [1m])` computes the per-second rate of change over a 1-minute window.
3. **Aggregation:** `avg()` aggregates across all CPU cores.
4. **Normalization:** Multiplying by 100 converts the fraction into a percentage.
5. **Active CPU Derivation:** Subtracting from 100 yields active CPU utilization.

8.4 Mathematical Interpretation

Let $I(t)$ denote idle CPU fraction, then:

$$CPUUsage = 100 \times (1 - I(t))$$

Insight: Monitoring idle time instead of active time avoids double counting across CPU modes and provides a more stable utilization metric.

8.5 Practical Considerations

- Short windows (e.g., 1m) provide responsiveness but may introduce noise
- Longer windows smooth fluctuations but delay scaling decisions

9 Automation via Prometheus API

```
requests.get("http://localhost:9090/api/v1/query")
```

Prometheus API enables external automation systems to consume metrics programmatically.

9.1 Automation Sequence Flow

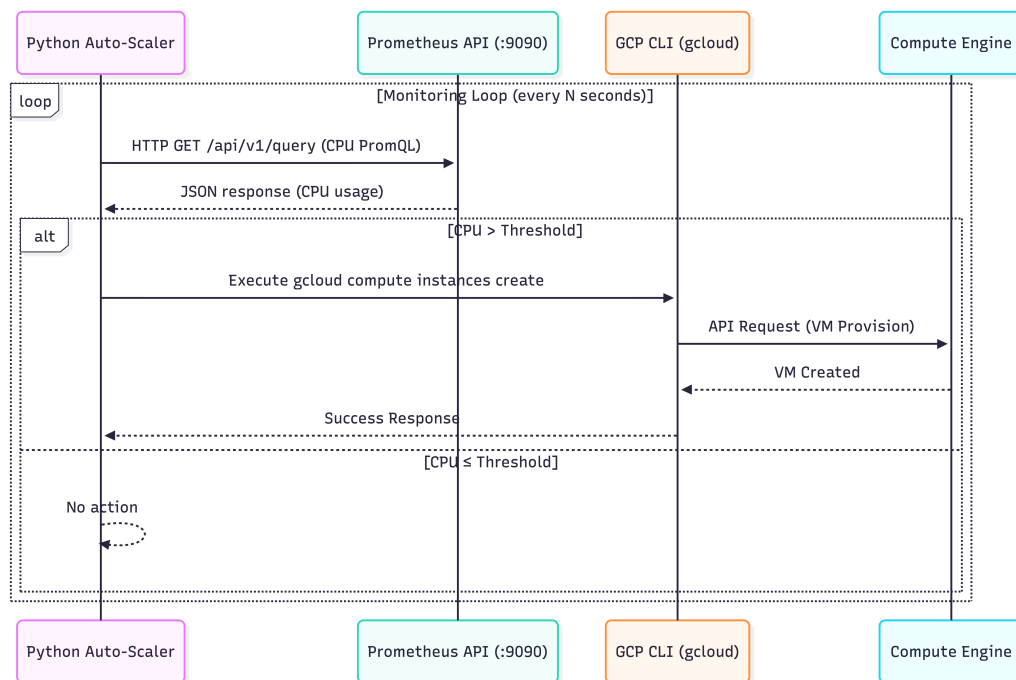


Figure 3: Sequence Diagram for Automated Scaling via Prometheus API

Insight: The system continuously polls Prometheus, evaluates CPU utilization, and conditionally triggers infrastructure provisioning, forming a closed-loop control system.

10 Application Deployment

```
apt install -y nginx
systemctl start nginx
```

Nginx confirms successful provisioning and deployment.

11 Results

- Accurate CPU monitoring achieved
- Threshold detection validated
- Automated VM creation successful
- End-to-end pipeline functional

12 Conclusion

The system demonstrates a complete auto-scaling pipeline integrating observability, decision-making, and infrastructure provisioning. It reflects real-world cloud-native architectures and provides a foundation for further enhancements such as alert-based scaling and visualization dashboards.

13 Plagiarism Declaration

I hereby declare that this submission is my original work and does not contain any plagiarized content.